

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

1. EXECUTIVE SUMMARY

Cofixer is a modern AI Agency & Technology full-stack web application with three parts: (1) a unified Frontend React 19 SPA with SSR for public pages, serving both the dynamic public website AND the admin dashboard CMS, (2) a NestJS v11.1.21 REST API Backend, and (3) PostgreSQL 18 + Redis 8 data layer. The public frontend is FULLY DYNAMIC - all content (services, projects, blog posts, team members, testimonials, FAQs, SEO metadata) is loaded from the API and rendered server-side for SEO. The admin dashboard (/dashboard) acts as a complete CMS where authenticated users can create, edit, and manage all frontend content in real-time. Authentication uses AES-256-GCM encrypted JWT in httpOnly cookies with CSRF protection. No tokens are ever stored in localStorage. Fully responsive and production-ready.

2. TECHNOLOGY STACK

Part A: Frontend + Dashboard (React 19 with SSR)

- Library: React 19.2.6
- Framework: React Router 7 with SSR (Server-Side Rendering for public pages)
- Build Tool: Vite 6.x
- Styling: Tailwind CSS 4.x
- Routing: React Router v7 (declarative routes, public pages SSR, dashboard client-only)
- State Management: Zustand 5.x (auth, csrf, ui state)
- Data Fetching: React Router 7 loaders (SSR data fetching) + TanStack Query v5 (client cache)
- Forms: React Hook Form + Zod validation
- Animations: Framer Motion
- Icons: Lucide React
- Charts: Recharts (dashboard analytics)

Part B: Backend API (Standalone NestJS Service)

- Framework: NestJS v11.1.21 (Node.js 22 LTS)
- Language: TypeScript 5.x (strict mode)
- API Style: RESTful JSON API with versioning (/api/v1/)
- Validation: class-validator + class-transformer
- Documentation: Swagger / OpenAPI 3.0 with @ApiSwagger decorator
- Authentication: AES-256-GCM encrypted JWT in httpOnly cookies + CSRF Double-Submit Cookie
- Authorization: RBAC with composite decorators (@AdminOnly, @RequireRoles, etc.)
- I18n: I18nHelper for ALL user-facing messages (never hardcoded)
- File Upload: Multer / AWS S3 (or local storage for dev)

Part C: Database & Infrastructure

- Primary DB: PostgreSQL 18
- ORM: TypeORM 0.3.x with @nestjs/typeorm
- Migrations: TypeORM CLI (never synchronize in production)
- Cache, Sessions & CSRF: Redis 8
- Containerization: Docker + Docker Compose

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

3. SYSTEM ARCHITECTURE

Three-Part Architecture

Cofixer is architected as three components: ONE frontend app with SSR for public SEO pages and SPA for the admin CMS dashboard, ONE backend API, and ONE database layer. All frontend content is DYNAMIC - loaded from the API. There is no hardcoded content. The dashboard is a full CMS where admins manage every aspect of the public website.

Part A: Frontend + Dashboard (Unified React with SSR)

- Public Pages (/ , /about, /services, etc.): Server-side rendered (SSR) for SEO
- Data Loading: React Router 7 loaders fetch API data BEFORE rendering HTML
- Meta Tags: Dynamic <title>, <meta>, JSON-LD rendered on server per request
- Dashboard (/dashboard/*): Client-side SPA with auth guards, no SSR needed
- All Content Dynamic: Hero text, service cards, project gallery, blog posts, team profiles, testimonials, FAQs - everything comes from API/database
- Zero Hardcoded Content: No static text, no static images, no static links in frontend code

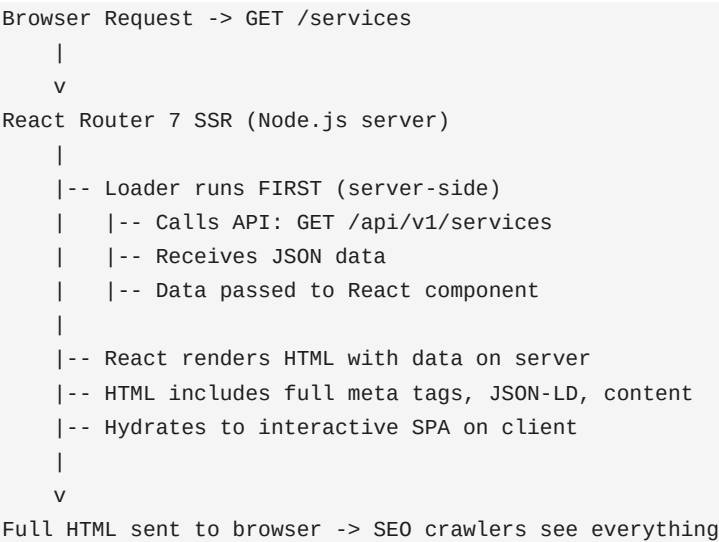
Part B: Backend API (NestJS)

- Standalone service exposing JSON REST APIs consumed by frontend loaders
- Cookie domain: Shared between frontend and API for httpOnly cookie auth
- CMS API Endpoints: Full CRUD for all content types (services, projects, blog, team, etc.)
- SEO API Endpoints: /api/v1/seo/settings, /api/v1/sitemap, /api/v1/robots-txt
- Base patterns: BaseEntity, BaseRepository, BaseService, BaseController for ALL features

Part C: Data Layer

- PostgreSQL 18: All relational data including CMS content and SEO settings
- Redis 8: Token blacklist, CSRF store, API cache, rate limiting

Request Flow (Public SSR Page)

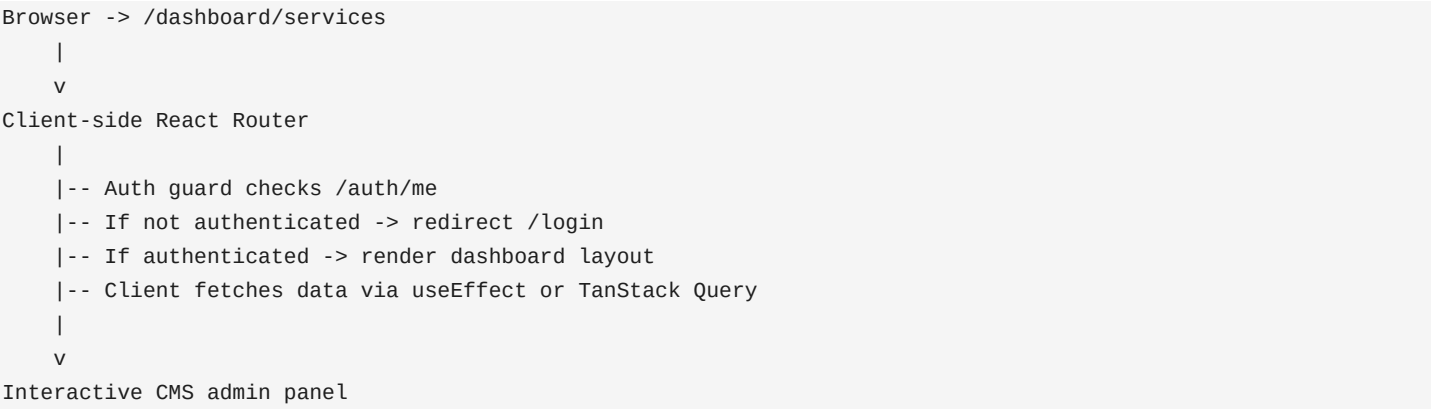


Request Flow (Dashboard SPA)

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026



COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

4. DYNAMIC FRONTEND ARCHITECTURE

100% Dynamic Content - Zero Hardcoded Data

The public website frontend contains NO hardcoded content. Every single piece of text, image, link, and meta tag is loaded from the backend API. The admin dashboard CMS controls everything. This makes Cofixer a true headless CMS architecture where the frontend is a presentation layer and the dashboard is the content management layer.

Dynamic Content Areas (All from API)

- Hero Section: Headline, subheadline, CTA buttons text/links, background image/video
- About Section: Company story text, statistics counters values, mission statement
- Services: Service cards (title, description, icon, image, order) - managed in CMS
- Projects: Portfolio grid (title, summary, images, category, client name) - managed in CMS
- Blog: Posts (title, excerpt, content, cover image, category, tags, author, publish date)
- Team: Member profiles (name, role, bio, image, social links) - managed in CMS
- Testimonials: Client reviews (name, role, company, content, rating, image)
- FAQs: Questions and answers organized by category - managed in CMS
- Contact: Form fields, Google Maps embed URL, office info - managed in CMS
- Footer: Links, copyright text, social media URLs - managed in CMS
- Navigation: Menu items, dropdown structure, links - managed in CMS

SEO Settings (Dynamic per Page)

- Page Title: Custom <title> for every route fetched from CMS SEO settings
- Meta Description: Custom <meta name="description"> per page
- Meta Keywords: <meta name="keywords"> per page
- OG Tags: Open Graph title, description, image for social sharing
- Twitter Cards: twitter:title, twitter:description, twitter:image
- Canonical URL: <link rel="canonical"> per page
- Robots Meta: <meta name="robots"> (index/noindex) per page
- Structured Data: JSON-LD schema (Organization, Service, Article, BreadcumbList, FAQPage)
- Hreflang: Multi-language support tags if needed
- Custom Head Scripts: Google Analytics, Tag Manager, custom tracking codes

SSR Data Loading with React Router 7 Loaders

```
// app/routes/public.routes.ts
import { route, index } from '@react-router/dev/routes';

export const publicRoutes = [
  index('pages/public/home.tsx'),
  route('about', 'pages/public/about.tsx'),
  route('services', 'pages/public/services.tsx'),
  route('services/:slug', 'pages/public/service-detail.tsx'),
  route('projects', 'pages/public/projects.tsx'),
  route('projects/:slug', 'pages/public/project-detail.tsx'),
  route('blog', 'pages/public/blog.tsx'),
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

```
route('blog/:slug', 'pages/public/blog-detail.tsx'),
route('team', 'pages/public/team.tsx'),
route('team/:id', 'pages/public/team-detail.tsx'),
route('testimonials', 'pages/public/testimonials.tsx'),
route('faqs', 'pages/public/faqs.tsx'),
route('contact', 'pages/public/contact.tsx'),
route('gallery', 'pages/public/gallery.tsx'),
route('sitemap.xml', 'pages/public/sitemap.tsx'), // Dynamic sitemap
route('robots.txt', 'pages/public/robots.tsx'), // Dynamic robots.txt
];

// app/pages/public/services.tsx
import { useLoaderData } from 'react-router';
import { serviceService } from '~/services/httpServices/serviceService';
import { seoService } from '~/services/httpServices/seoService';

// SERVER-SIDE LOADER - runs before render
export async function loader() {
  const [services, seoSettings] = await Promise.all([
    serviceService.getAll(),
    seoService.getPageSettings('services'),
  ]);

  return { services, seoSettings };
}

export default function ServicesPage() {
  const { services, seoSettings } = useLoaderData<typeof loader>();

  return (
    <>
      <DynamicMetaTags seo={seoSettings} />
      <StructuredData type="Service" data={services} />
      <HeroSection
        title={seoSettings.heroTitle}
        subtitle={seoSettings.heroSubtitle}
      />
      <ServicesGrid services={services} />
    </>
  );
}
```

DynamicMetaTags Component

```
// app/components/seo/DynamicMetaTags.tsx
import { useEffect } from 'react';
import { useLoaderData } from 'react-router';

interface SeoSettings {
  title: string;
  description: string;
  keywords: string;
  ogTitle: string;
  ogDescription: string;
}
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

```
ogImage: string;
canonicalUrl: string;
robots: string;
jsonLd: object;
}

export function DynamicMetaTags({ seo }: { seo: SeoSettings }) {
  // On server: React Helmet renders to HTML head
  // On client: updates document.head after hydration
  return (
    <>
      <title>{seo.title}</title>
      <meta name="description" content={seo.description} />
      <meta name="keywords" content={seo.keywords} />
      <meta property="og:title" content={seo.ogTitle} />
      <meta property="og:description" content={seo.ogDescription} />
      <meta property="og:image" content={seo.ogImage} />
      <meta name="twitter:title" content={seo.ogTitle} />
      <meta name="twitter:description" content={seo.ogDescription} />
      <meta name="twitter:image" content={seo.ogImage} />
      <link rel="canonical" href={seo.canonicalUrl} />
      <meta name="robots" content={seo.robots} />
      <script type="application/ld+json">
        {JSON.stringify(seo.jsonLd)}
      </script>
    </>
  );
}
```

Dynamic Sitemap & Robots.txt

```
// API endpoint: GET /api/v1/sitemap
// Returns XML sitemap generated from all published content

// API endpoint: GET /api/v1/robots-txt
// Returns robots.txt content from CMS settings

// React Router routes:
// /sitemap.xml -> fetches from API and returns XML response
// /robots.txt -> fetches from API and returns plain text
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

5. ADMIN DASHBOARD (FULL CMS)

Dashboard = Content Management System

The /dashboard area is a complete CMS that allows authenticated admin/editor users to manage EVERY piece of content that appears on the public website. No code changes are needed to update the frontend. All changes are reflected immediately on the public site.

CMS Management Areas

- Dashboard Overview: Stats (total services, projects, blog posts, contacts, views)
- Services CMS: Create, edit, delete, reorder, feature/unfeature services
- Projects CMS: Create, edit, delete, upload images/gallery, set featured, categories
- Blog CMS: Create, edit, delete, publish/unpublish, set author, categories, tags
- Team CMS: Create, edit, delete team members, upload photos, set social links
- Testimonials CMS: Create, edit, delete, set ratings, feature/unfeature
- FAQs CMS: Create, edit, delete, organize by category, set order
- Contact Inbox: View, mark read/unread, delete contact form submissions
- Media Library: Upload, organize, delete images and assets used across the site

SEO Management

- Global SEO Settings: Site title template, default meta description, OG image, favicon
- Per-Page SEO: Edit title, meta description, keywords, OG tags for every route
- Structured Data: Manage JSON-LD schemas per page type
- Sitemap Settings: Configure changefreq, priority per content type
- Robots.txt Editor: Edit robots.txt content directly in dashboard
- Analytics Integration: Add Google Analytics ID, Tag Manager ID, custom scripts
- Social Links: Manage social media URLs for footer and team members
- Navigation Editor: Add, edit, reorder, delete menu items and dropdowns

Homepage/CMS Content Editor

- Hero Editor: Edit headline, subheadline, CTA button text and link, background image/video
- Stats Editor: Edit counter values and labels (e.g., 298+ Projects Delivered)
- About Editor: Edit company story, mission, vision text
- CTA Sections: Edit call-to-action text and buttons throughout the site
- Footer Editor: Edit copyright text, footer links columns, newsletter signup text
- Theme Settings: Primary colors, font choices, spacing (if supported by template)

Dashboard Pages Structure

```
app/pages/dashboard/  
|-- layout.tsx           # Sidebar + auth guard  
|-- index.tsx            # Overview stats + charts  
|  
|-- services/  
|   |-- index.tsx        # Service list (table with sort/filter)  
|   |-- create.tsx       # Create service form
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

```
| |-- edit.tsx          # Edit service form
|
|-- projects/
| |-- index.tsx        # Project list
| |-- create.tsx       # Create project (with image upload)
| |-- edit.tsx         # Edit project
|
|-- blog/
| |-- index.tsx        # Blog post list (draft/published filter)
| |-- create.tsx       # Create post (rich text editor)
| |-- edit.tsx         # Edit post
|
|-- team/
| |-- index.tsx        # Team member list
| |-- create.tsx       # Create member
| |-- edit.tsx         # Edit member
|
|-- testimonials/
| |-- index.tsx        # Testimonial list
| |-- create.tsx       # Create testimonial
|
|-- faqs/
| |-- index.tsx        # FAQ list grouped by category
| |-- create.tsx       # Create FAQ
|
|-- contacts/
| |-- index.tsx        # Contact submissions inbox
| |-- detail.tsx       # View single message
|
|-- media/
| |-- index.tsx        # Media library (upload grid)
|
|-- settings/
| |-- seo.tsx          # Global and per-page SEO settings
| |-- navigation.tsx   # Menu editor
| |-- general.tsx      # Site name, logo, favicon, theme colors
| |-- scripts.tsx      # Custom head/footer scripts
```

Rich Text Editor for Content

- Blog posts: Rich text editor (TipTap or Lexical) with image insertion
- Service descriptions: Rich text with formatting
- Project content: Rich text editor for case study content
- Team bio: Rich text editor for formatted bios

Image Upload & Media Management

- Drag-and-drop image upload in all create/edit forms
- Image preview before saving
- Gallery management for projects (reorder, delete, add images)
- Media library with folders/tags for organization
- Automatic image optimization (WebP conversion, resizing) on upload

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

6. SEO ARCHITECTURE (Server-Side Rendered)

Why SSR is Required

For the public website to be SEO-friendly and rank on search engines, it MUST serve pre-rendered HTML with all content and meta tags. Search engine crawlers cannot execute JavaScript to wait for client-side data fetching. React Router 7 SSR solves this by rendering HTML on the server with all dynamic content already included.

SSR Strategy

- Public Pages: SSR with React Router 7 loaders - HTML rendered server-side
- Dashboard: Client-side only - no SSR needed (not indexed by search engines)
- Data Fetching: Loaders fetch API data BEFORE React renders anything
- Meta Tags: Rendered in <head> on server, visible to crawlers immediately
- Structured Data: JSON-LD injected server-side in HTML

Dynamic SEO Per Route

```
// Every public page has a loader that fetches SEO settings
export async function loader({ params }: LoaderFunctionArgs) {
  const slug = params.slug;

  const [content, seoSettings] = await Promise.all([
    // Fetch the main content
    serviceService.getBySlug(slug),
    // Fetch SEO settings for this specific page
    seoService.getPageSettings(`service/${slug}`),
  ]);

  // If no specific SEO, use content fields as fallback
  const seo = {
    title: seoSettings?.title || content.title,
    description: seoSettings?.description || content.excerpt,
    ogImage: seoSettings?.ogImage || content.image,
    canonicalUrl: `https://cofixer.com/services/${slug}`,
    jsonLd: generateServiceSchema(content),
  };

  return { content, seo };
}
```

Dynamic JSON-LD Structured Data

```
// Generated per page based on CMS content

// Homepage - Organization schema
{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "Cofixer",
  "url": "https://cofixer.com",
  "logo": "https://cofixer.com/logo.png",
  "description": "AI Agency & Technology",
}
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

```
"sameAs": ["https://twitter.com/cofixer", "https://linkedin.com/company/cofixer"]
}

// Service page - Service schema
{
  "@context": "https://schema.org",
  "@type": "Service",
  "name": "AI Strategy & Consulting",
  "provider": { "@type": "Organization", "name": "Cofixer" },
  "description": "We assess your business and identify high-impact AI opportunities..."
}

// Blog post - Article schema
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "Post Title",
  "author": { "@type": "Person", "name": "Author Name" },
  "datePublished": "2024-01-15",
  "image": "https://cofixer.com/images/post.jpg"
}

// FAQ page - FAQPage schema
{
  "@context": "https://schema.org",
  "@type": "FAQPage",
  "mainEntity": [
    {
      "@type": "Question",
      "name": "What services does Cofixer offer?",
      "acceptedAnswer": {
        "@type": "Answer",
        "text": "Cofixer offers AI strategy, integration, and custom solutions..."
      }
    }
  ]
}
```

Dynamic Sitemap Generation

```
// Backend API generates sitemap from all published content
GET /api/v1/sitemap

Response (XML):
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://cofixer.com/</loc>
    <lastmod>2024-01-15</lastmod>
    <changefreq>daily</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://cofixer.com/services/ai-strategy</loc>
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

```
<lastmod>2024-01-14</lastmod>
<changefreq>weekly</changefreq>
<priority>0.8</priority>
</url>
<url>
  <loc>https://cofixer.com/blog/post-slug</loc>
  <lastmod>2024-01-10</lastmod>
  <changefreq>monthly</changefreq>
  <priority>0.6</priority>
</url>
<!-- Auto-generated from all services, projects, blog posts, team, etc. -->
</urlset>
```

SEO Checklist

- Server-rendered HTML with all content visible without JavaScript
- Unique <title> and <meta description> for every page
- Open Graph tags for social sharing
- Twitter Card meta tags
- Canonical URLs to prevent duplicate content
- /sitemap.xml auto-generated from database
- /robots.txt configurable from CMS
- Structured data (JSON-LD) for Organization, Service, Article, FAQPage, BreadcrumbList
- Semantic HTML5 tags (header, nav, main, article, section, footer)
- Alt tags on all images (from CMS)
- URL slugs for all content (SEO-friendly)
- Breadcrumb navigation with schema markup
- Page load time < 3 seconds
- Mobile-responsive design
- HTTPS with valid SSL certificate

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

7. API DESIGN

Response Format (From guides)

```
{
  "success": true,
  "statusCode": 200,
  "message": "Services retrieved successfully",
  "data": [...],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 50,
    "totalPages": 5
  },
  "timestamp": "2024-11-02T10:30:00.000Z",
  "path": "/api/v1/services"
}
```

Public Endpoints

GET	/api/v1/auth/csrf-token	
GET	/api/v1/services	-> List all services
GET	/api/v1/services/:slug	-> Single service detail
GET	/api/v1/projects	-> List projects
GET	/api/v1/projects/:slug	-> Single project detail
GET	/api/v1/blog	-> List published posts
GET	/api/v1/blog/:slug	-> Single post detail
GET	/api/v1/team	-> List team members
GET	/api/v1/team/:id	-> Single team member
GET	/api/v1/testimonials	-> List testimonials
GET	/api/v1/faqs	-> List FAQs by category
POST	/api/v1/contact	-> Submit contact form (CSRF required)
GET	/api/v1/stats	-> Public stats (counters)
GET	/api/v1/sitemap	-> Dynamic XML sitemap
GET	/api/v1/robots-txt	-> Dynamic robots.txt
GET	/api/v1/seo/page/:route	-> SEO settings for a specific route

Auth Endpoints

POST	/api/v1/auth/register	-> Create account (CSRF required)
POST	/api/v1/auth/login	-> Login (sets encrypted httpOnly cookies)
POST	/api/v1/auth/logout	-> Clear cookies, blacklist refresh token
POST	/api/v1/auth/refresh	-> Rotate refresh token
GET	/api/v1/auth/me	-> Get current user

Admin/CMS Endpoints (JWT + CSRF + Admin/Editor role)

POST	/api/v1/admin/services	-> Create service
PUT	/api/v1/admin/services/:id	-> Update service
DELETE	/api/v1/admin/services/:id	-> Delete service
PATCH	/api/v1/admin/services/:id/feature	-> Toggle featured
PATCH	/api/v1/admin/services/reorder	-> Reorder services

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

POST	/api/v1/admin/projects	-> Create project
PUT	/api/v1/admin/projects/:id	-> Update project
DELETE	/api/v1/admin/projects/:id	-> Delete project
POST	/api/v1/admin/blog	-> Create blog post
PUT	/api/v1/admin/blog/:id	-> Update post
DELETE	/api/v1/admin/blog/:id	-> Delete post
PATCH	/api/v1/admin/blog/:id/publish	-> Toggle publish status
POST	/api/v1/admin/team	-> Create team member
PUT	/api/v1/admin/team/:id	-> Update team member
DELETE	/api/v1/admin/team/:id	-> Delete team member
POST	/api/v1/admin/testimonials	-> Create testimonial
PUT	/api/v1/admin/testimonials/:id	-> Update
DELETE	/api/v1/admin/testimonials/:id	-> Delete
POST	/api/v1/admin/faqs	-> Create FAQ
PUT	/api/v1/admin/faqs/:id	-> Update FAQ
DELETE	/api/v1/admin/faqs/:id	-> Delete FAQ
GET	/api/v1/admin/contacts	-> List contact messages
PUT	/api/v1/admin/contacts/:id/read	-> Mark as read
DELETE	/api/v1/admin/contacts/:id	-> Delete message
POST	/api/v1/admin/media/upload	-> Upload image/asset
DELETE	/api/v1/admin/media/:id	-> Delete asset
GET	/api/v1/admin/seo/settings	-> Get all SEO settings
PUT	/api/v1/admin/seo/settings	-> Update SEO settings
GET	/api/v1/admin/seo/page/:route	-> Get SEO for specific route
PUT	/api/v1/admin/seo/page/:route	-> Update SEO for specific route
GET	/api/v1/admin/navigation	-> Get navigation menu
PUT	/api/v1/admin/navigation	-> Update navigation menu
GET	/api/v1/admin/settings/general	-> Get general site settings
PUT	/api/v1/admin/settings/general	-> Update general settings

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

8. AUTHENTICATION & SECURITY

No Client-Side Token Storage

- localStorage: NEVER used for auth tokens or session data
- sessionStorage: NEVER used for auth tokens or session data
- IndexedDB: NEVER used for auth tokens or session data
- Cookies: Only httpOnly Secure cookies managed by the browser

Global JWT Guard + @Public() Decorator

```
// app.module.ts
providers: [
  {
    provide: APP_GUARD,
    useClass: JwtAuthGuard, // Global: all routes protected by default
  },
],

// Public routes use @Public() decorator
@Public()
@Get('services')
async findAll() { ... }

// Admin routes protected by default + RBAC
@AdminOnly()
@Post('admin/services')
async create(...) { ... }
```

RBAC with Composite Decorators

```
@Public()           -> Bypass authentication
@AdminOnly()         -> Admin role required
@adminOrOwner()      -> Admin OR resource owner
@ModeratorOrAdmin()  -> Moderator or Admin
@RequireRoles(...)   -> Custom role combination

// Editor can manage content but not users/settings
@RequireRoles(RolesEnum.ADMIN, RolesEnum.EDITOR)
@Post('admin/blog')
async createPost(...) { ... }
```

Encrypted httpOnly Cookie Authentication

- Algorithm: AES-256-GCM authenticated encryption
- Cookie Format: base64(iv + ciphertext + auth_tag)
- Backend decrypts cookie on every request before JWT validation
- Even if cookie is stolen, attacker cannot use it without encryption key

CSRF Double-Submit Cookie Pattern

```
GET /api/v1/auth/csrf-token
-> Sets csrf_token cookie + returns token in body
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

POST /api/v1/admin/blog

Headers: X-CSRF-Token: <token-from-body>

Cookies: access_token=<encrypted>; csrf_token=<same-token>

Backend validates:

1. Decrypt access_token -> validate JWT
2. X-CSRF-Token header == csrf_token cookie value
3. Check Redis for token validity + fingerprint match

Security Measures

- helmet: CSP, HSTS, X-Frame-Options, X-Content-Type-Options
- cors: Whitelist frontend origin only, credentials: true
- rate-limit: 100 req/min per IP (Redis-backed)
- bcrypt: Password hashing (salt rounds: 12)
- TypeORM: Parameterized queries prevent SQL injection
- File Upload: MIME validation, 5MB limit, image optimization
- Content Security Policy: Restrict inline scripts
- I18nHelper: ALL user-facing messages use translation keys

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

9. DATABASE DESIGN (TypeORM)

New Entity: SeoSettings

```
// src/database/entities/seo-settings.entity.ts
@Entity('seo_settings')
export class SeoSettings extends BaseEntity {
  @Column()
  route: string; // e.g., '/', '/services', '/blog/:slug'

  @Column()
  pageType: string; // 'home', 'services', 'service_detail', 'blog', etc.

  @Column({ nullable: true })
  title: string;

  @Column('text', { nullable: true })
  metaDescription: string;

  @Column('simple-array', { nullable: true })
  metaKeywords: string[];

  @Column({ nullable: true })
  ogTitle: string;

  @Column('text', { nullable: true })
  ogDescription: string;

  @Column({ nullable: true })
  ogImage: string;

  @Column({ nullable: true })
  twitterImage: string;

  @Column({ nullable: true })
  canonicalUrl: string;

  @Column({ default: 'index, follow' })
  robotsMeta: string;

  @Column('jsonb', { nullable: true })
  jsonLdSchema: object; // Structured data JSON

  @Column({ nullable: true })
  customHeadScripts: string; // Analytics, tracking codes
}

// src/database/entities/site-settings.entity.ts
@Entity('site_settings')
export class SiteSettings extends BaseEntity {
  @Column({ default: 'Cofixer' })
  siteName: string;
```


COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

```
@Column({ nullable: true })
logo: string;

@Column({ nullable: true })
favicon: string;

@Column('text', { nullable: true })
copyrightText: string;

@Column('jsonb', { nullable: true })
socialLinks: { twitter?: string; linkedin?: string; github?: string };

@Column('jsonb', { nullable: true })
themeColors: { primary?: string; accent?: string };

@Column({ nullable: true })
googleAnalyticsId: string;

@Column({ nullable: true })
googleTagManagerId: string;

@Column('text', { nullable: true })
customScripts: string;
}
```

New Entity: NavigationMenu

```
// src/database/entities/navigation-menu.entity.ts
@Entity('navigation_menus')
export class NavigationMenu extends BaseEntity {
  @Column()
  name: string; // 'main', 'footer', 'sidebar'

  @Column('jsonb')
  items: NavigationItem[];

  @Column({ default: true })
  isActive: boolean;
}

interface NavigationItem {
  id: string;
  label: string;
  url: string;
  icon?: string;
  children?: NavigationItem[];
  isExternal?: boolean;
  order: number;
}
```

Updated: ContactMessage

```
// Added fields for better CMS management
@Entity('contact_messages')
export class ContactMessage extends BaseEntity {
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

```
@Column()  
name: string;  
  
@Column()  
email: string;  
  
@Column({ nullable: true })  
phone: string;  
  
@Column()  
subject: string;  
  
@Column('text')  
message: string;  
  
@Column({ default: false })  
read: boolean;  
  
@Column({ nullable: true })  
notes: string; // Admin notes about this message  
  
@Column({ default: 'new' })  
status: 'new' | 'in_progress' | 'resolved' | 'spam';  
}
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

10. REDIS 8 INTEGRATION

Cache Strategy for Dynamic Content

- Public API responses cached aggressively (services, projects, blog, team)
- Cache invalidation on ANY CMS update (admin mutation triggers cache clear)
- SEO settings cached with short TTL (60s) to reflect dashboard changes quickly
- Sitemap cached with medium TTL (5 min)

Redis Key Patterns

cofixer:cache:services:all	-> Service[] (TTL: 300s)
cofixer:cache:services:slug:{slug}	-> Service (TTL: 300s)
cofixer:cache:projects:all	-> Project[] (TTL: 300s)
cofixer:cache:projects:slug:{slug}	-> Project (TTL: 300s)
cofixer:cache:blog:published	-> BlogPost[] (TTL: 300s)
cofixer:cache:blog:slug:{slug}	-> BlogPost (TTL: 300s)
cofixer:cache:team:all	-> TeamMember[] (TTL: 300s)
cofixer:cache:testimonials:all	-> Testimonial[] (TTL: 300s)
cofixer:cache:faqs:all	-> Faq[] (TTL: 300s)
cofixer:cache:seo:route:{route}	-> SeoSettings (TTL: 60s)
cofixer:cache:sitemap	-> XML string (TTL: 300s)
cofixer:cache:stats:public	-> Stats object (TTL: 300s)
cofixer:refresh-blacklist:{jti}	-> "revoked"
cofixer:csrf:{token}	-> CSRF data

Cache Invalidation on CMS Updates

```
// When admin updates a service:
@Patch(':id')
@CacheInvalidate({ patterns: ['services:', 'sitemap', 'stats:public'] })
async update(...) { ... }

// When admin publishes a blog post:
@Post()
@CacheInvalidate({ patterns: ['blog:', 'sitemap', 'stats:public'] })
async create(...) { ... }

// When admin updates SEO settings:
@Put('seo/page/:route')
@CacheInvalidate({ patterns: ['seo:'] })
async updateSeo(...) { ... }
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

11. DEVELOPMENT ENVIRONMENT

Docker Compose

```
# docker-compose.dev.yml
version: '3.8'

services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_DB: cofixer
      POSTGRES_USER: cofixer
      POSTGRES_PASSWORD: cofixer_dev
    ports: ["5432:5432"]
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./cofixer-api/src/database/migrations:/migrations

  redis:
    image: redis:8-alpine
    ports: ["6379:6379"]

  api:
    build: ./cofixer-api
    ports: ["3001:3001"]
    env_file: ./cofixer-api/.env
    depends_on: [db, redis]
    volumes:
      - ./cofixer-api:/app
      - /app/node_modules
    command: npm run start:dev

  web:
    build: ./cofixer-web
    ports: ["5173:5173"]
    env_file: ./cofixer-web/.env
    environment:
      - VITE_API_URL=http://localhost:3001/api/v1
    depends_on: [api]
    volumes:
      - ./cofixer-web:/app
      - /app/node_modules
    command: npm run dev

volumes:
  pgdata:
```

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

12. PRODUCTION DEPLOYMENT & SEO

Nginx Configuration

```
server {
  listen 443 ssl http2;
  server_name cofixer.com www.cofixer.com;

  ssl_certificate /etc/letsencrypt/live/cofixer.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/cofixer.com/privkey.pem;

  # Frontend (SSR capable)
  location / {
    # React Router 7 SSR server or static build
    proxy_pass http://localhost:5173/;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }

  # Backend API
  location /api/ {
    proxy_pass http://localhost:3001/;
    proxy_cookie_path / "/; Secure; SameSite=None";
  }

  # Sitemap and robots (serve directly or proxy)
  location = /sitemap.xml {
    proxy_pass http://localhost:3001/api/v1/sitemap;
  }

  location = /robots.txt {
    proxy_pass http://localhost:3001/api/v1/robots-txt;
  }

  # Static uploads
  location /uploads/ {
    alias /var/www/cofixer-api/uploads/;
    expires 7d;
    add_header Cache-Control "public, immutable";
  }
}
```

SEO Production Checklist

- Server-rendered HTML for all public routes (view page source shows full content)
- Dynamic <title> and <meta> tags in <head> for every page
- Open Graph tags populated from CMS data
- JSON-LD structured data injected server-side
- /sitemap.xml auto-generated and submitted to Google Search Console
- /robots.txt configurable from dashboard

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

- Canonical URLs set for all pages
- 301 redirects for changed slugs (stored in CMS)
- Image alt tags from CMS for all images
- Semantic HTML5 structure (header, nav, main, article, section)
- Breadcrumb navigation with schema markup
- Mobile-first responsive design
- Core Web Vitals: LCP < 2.5s, FID < 100ms, CLS < 0.1
- HTTPS with HSTS enabled
- Compress responses (gzip/brotli)
- Image optimization (WebP, responsive srcset)
- Lazy loading for below-fold images

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

13. PROJECT TIMELINE

Phase 1: Foundation (Week 1)

- Day 1-2: Init projects, Docker setup, Base patterns, database schema with SeoSettings
- Day 3-4: Auth module, encrypted cookies, CSRF, l18nHelper
- Day 5-7: Content API modules (Services, Projects, Blog, Team, Testimonials, FAQs)

Phase 2: Frontend Public + SSR (Week 2)

- Day 8-10: React Router 7 SSR setup, loaders, DynamicMetaTags, public pages
- Day 11-12: Dynamic sitemap, robots.txt, JSON-LD structured data
- Day 13-14: SEO API endpoints, SeoSettings entity, per-page SEO loading

Phase 3: Dashboard CMS (Week 3)

- Day 15-17: Admin layout, auth guards, dashboard overview stats
- Day 18-19: Content CRUD (Services, Projects, Blog with rich text editor)
- Day 20-21: Team, Testimonials, FAQs, Contact inbox CMS

Phase 4: CMS Polish + Media (Week 4)

- Day 22-23: Media library, image upload, drag-and-drop, image optimization
- Day 24-25: SEO management dashboard, navigation editor, general settings
- Day 26-28: Homepage content editor, CTA editor, footer editor

Phase 5: Testing & Launch (Week 5)

- Day 29-30: E2E tests, SEO audit, Lighthouse testing
- Day 31-32: Performance optimization, image optimization, caching fine-tuning
- Day 33-35: Production deployment, SSL, Google Search Console setup

14. ACCEPTANCE CRITERIA

- Public website is 100% dynamic - zero hardcoded content in frontend code
- All content (services, projects, blog, team, testimonials, FAQs) managed via dashboard CMS
- Dashboard CMS allows full CRUD for all content types with image uploads
- Public pages render server-side (SSR) - view page source shows full HTML content
- Dynamic <title> and <meta description> rendered server-side for every route
- Open Graph tags populated dynamically from CMS data
- JSON-LD structured data injected server-side on all relevant pages
- /sitemap.xml auto-generated from database content
- /robots.txt configurable from dashboard CMS
- SEO settings per page editable from dashboard (title, meta, OG tags, canonical)
- Navigation menu editable from dashboard CMS
- Homepage hero, stats, about text, CTAs editable from dashboard
- Footer content (copyright, links, social) editable from dashboard
- Media library for uploading and managing images/assets

COFIXER

Full-Stack Product Requirements Document (PRD)

Date: May 15, 2026

- Rich text editor for blog posts, service descriptions, project content
- JWT tokens NEVER stored in localStorage/sessionStorage
- AES-256-GCM encrypted httpOnly cookie authentication works end-to-end
- CSRF protection on all mutating CMS operations
- Lighthouse SEO score = 100
- Lighthouse Performance score > 90
- All public pages indexable by search engine crawlers (no JS required to see content)
- Mobile-responsive across all breakpoints
- Zero critical/high npm audit vulnerabilities